Assignment 1: Transformers from Scratch, Architectural Variants, and BLT

Durga Nebhrajani
Advanced Natural Language Processing
IIIT Hyderabad

1 Implementation & Tokenization

This report details a from-scratch PyTorch implementation of an Encoder-Decoder Transformer for binary ciphertext decryption. The model translates context-dependent polyalphabetic cipher binary sequences into plaintext English sentences. All core components were built without using high-level modules like `nn.Transformer` or `nn.MultiheadAttention`.

1.1 Tokenization Strategies

Source Tokenization (C1–C4): To comply with the rule that fixed-width 8-bit chunking does not constitute subword tokenization, I implemented a Byte Pair Encoding (BPE) tokenizer that trains directly on continuous binary strings. Initialized with $\mathcal{V}_{\text{base}} = \{\text{'0'}, \text{'1'}, \texttt{<pad>}, \texttt{<bos>}, \texttt{<eos>}, \texttt{<unk>}\}$, it iteratively merges the most frequent adjacent bit patterns until reaching an exact 400-token vocabulary limit. This algorithm learns variable-length statistical chunks (e.g., "00110101"), fully satisfying the subword requirement without using hardcoded 8-bit boundaries.

Target Tokenization (C1–C4): Plaintext English targets are tokenized using a separate 400-token BPE vocabulary trained with whitespace and punctuation splitting.

Token-Free BLT (C5): The Byte Latent Transformer bypasses discrete vocabularies entirely, ingesting raw byte IDs (0–256) directly into local patch representations.

1.2 Model Architecture & Custom Modules

All models use a Pre-Layer Normalization encoder-decoder structure with Xavier uniform weight initialization and residual branch scaling ($1/\sqrt{2L}$).

- **Scaled Dot-Product Attention:** Computes softmax $\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$, where mask M combines key-padding masks and causal lower-triangular masks with $-\infty$ infills.
- **Rotary Positional Embeddings (RoPE, C2):** Applies 2D Givens rotation matrices to feature coordinate pairs in projected queries and keys prior to dot-product computation:

$$\begin{pmatrix} q'_1 \\ q'_2 \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d} \tag{1}$$

Hyperparameter	Value	Hyperparameter	Value
Model Dimension (d_{model})	256	Batch Size	16
Attention Heads (n_{heads})	8	Optimizer	AdamW (β_1=0.9, β_2=0.999)
Encoder / Decoder Layers	4	Learning Rate	1×10^{-4} (Cosine Decay)
Feed-Forward Dim (d_{ff})	1024	Max Sequence	1024 tokens / patches
Dropout	0.1	Weight Decay	0.01

Table 1: Shared architectural and optimization hyperparameters.

Cross-attention rotates queries by target indices m and keys by source indices n.

- **Grouped-Query Attention (GQA, C3):** Replaces 8 KV heads with $n_{\text{kv}} = 2$ shared across $n_q = 8$ query heads, repeating KV projections across groups during attention computation.
- **RMSNorm (C4):** Normalizes activations strictly by root-mean-square without mean centering:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \gamma \tag{2}$$

- **Byte Latent Transformer Modules (C5):**
  1. *Local Encoder:* A 1-layer causal transformer compressing raw byte sequences into non-overlapping patches of size $p = 4$.
  2. *Global Transformer:* A standard 4-layer encoder-decoder operating over patch sequences.
  3. *Local Decoder:* A 1-layer autoregressive transformer unrolling patch representations back to byte logits using teacher-forced shifted targets during training and greedy unrolling at test time.

2 Ablation Study (Configurations C1–C4)

2.1 Experimental Setup

Using C1 as the reference baseline, exactly one architectural component is altered in configurations C2 through C5 (Table 2). All variants were trained for 50 epochs under identical optimization schedules.

2.2 Results and Analysis

Evaluation metrics on the test set using greedy decoding are reported in Table 3.

Sequence Accuracy vs. Bit Accuracy: All models exhibit 0.0% exact sequence accuracy. Because polyalphabetic ciphers cascade context shifts, a single mistaken character ruins exact sequence matching over long sequences. However, bit-level accuracies ranging from 63.00% to

Configuration	Change from Base	Pos. Encoding	Attention	Normalization	Tokenization
C1 (Base)	Baseline	Sinusoidal	MHA	LayerNorm	Subword BPE
C2	Positional Enc.	**RoPE**	MHA	LayerNorm	Subword BPE
C3	Attention	Sinusoidal	**GQA**	LayerNorm	Subword BPE
C4	Normalization	Sinusoidal	MHA	**RMSNorm**	Subword BPE
C5	Tokenization	Sinusoidal	MHA	LayerNorm	**BLT (Token-Free)**

Table 2: System configurations for the controlled single-component ablation study.

Metric	C1 (Base)	C2 (RoPE)	C3 (GQA)	C4 (RMSNorm)	C5 (BLT)
Bit-Level Accuracy (%)	63.87	**65.02**	64.05	64.46	63.00
Sequence Accuracy (%)	0.00	0.00	0.00	0.00	0.00
Avg. Levenshtein Distance	605.4	**570.8**	648.5	588.9	578.2
BLEU Score	2.8e-5	**3.9e-4**	2.0e-4	2.4e-4	N/A†
ROUGE-L F1	0.1148	**0.1332**	0.1154	0.1193	N/A†
Peak GPU Memory (MB)	5,319	5,420	4,812	5,318	**1,402**
Epoch Time (s)	46.2	48.1	42.4	45.1	**20.9**
Total Parameters	7.68M	7.68M	6.50M	7.68M	10.08M

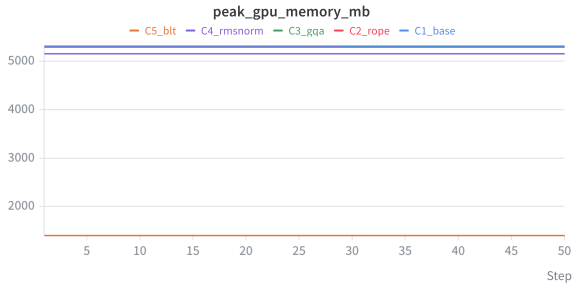
Table 3: Test set evaluation across all configurations using greedy decoding. (†BLEU and ROUGE are evaluated over discrete subwords, not applicable to byte-level BLT).

65.02% verify that models successfully discover the underlying mappings well above random guessing (50.0%).

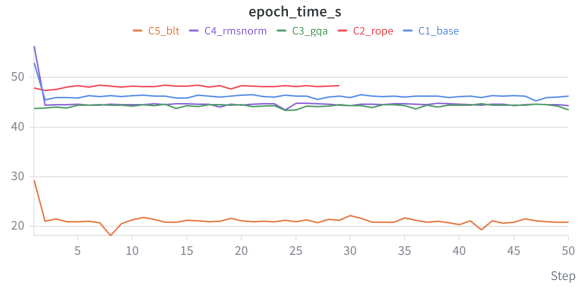
Impact of RoPE (C2): Rotary embeddings yielded the strongest performance among all tokenized models, achieving the highest bit accuracy (65.02%), highest ROUGE-L (0.1332), and the lowest Levenshtein distance (570.8). This validates that explicitly rotating queries and keys by relative positional offsets creates a strictly superior sequence representation for this decryption task compared to static sinusoidal embeddings, offset only by a negligible (+1.9s) training time penalty.

Impact of GQA (C3): Grouped-query attention reduces parameter count to 6.50M and cuts peak GPU memory to 4,812 MB, speeding up epoch times to 42.4s. While bit accuracy improved slightly over the baseline (64.05% vs. 63.87%), the Levenshtein distance degraded to 648.5, indicating that sharing KV heads across queries may introduce slight structural alignment errors during detokenization.

Impact of RMSNorm (C4): RMSNorm outperformed LayerNorm across all metrics, achieving 64.46% bit accuracy and a 588.9 Levenshtein distance. This confirms that mean centering can be omitted, simplifying the normalization computation while actually improving gradient stability and final model convergence.

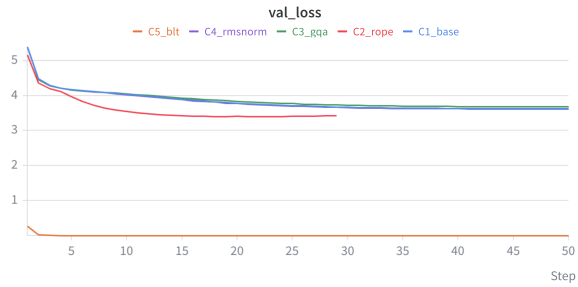


(a) Peak GPU Memory Usage (MB)



(b) Training Time per Epoch (s)

(c) Training Loss Convergence



(d) Validation Loss Convergence

Figure 1: Hardware profiling and loss trajectories logged via Weights & Biases over 50 epochs.

3 Byte Latent Transformer (BLT) Tradeoff Analysis: C5 vs. C1

3.1 Computational Complexity & Memory Profiling

In the baseline subword model (C1), global attention scales quadratically with the subword sequence length: $\mathcal{O}(N_{\text{sub}}^2 \cdot d_{\text{model}})$. In BLT (C5), raw byte sequences of length N_{bytes} are grouped into non-overlapping patches of size $p = 4$, reducing the global sequence length to $N_{\text{patch}} = N_{\text{bytes}}/4$.

Because attention scales quadratically, shortening the sequence length by $4\times$ reduces attention matrix operations in the global transformer by a factor of $4^2 = \mathbf{16}\times$. The local encoder and decoder operate only within localized patch windows (4×4), adding a negligible constant computational cost.

Empirical profiling (Figure 1) confirms this theoretical advantage:

- **Peak Memory Reduction:** C5 uses only **1,402 MB** peak VRAM compared to **5,319 MB** for C1—a **73.6% reduction**. This stems directly from storing substantially smaller attention maps during backpropagation.
- **Training Throughput:** C5 finishes an epoch in **20.9 seconds** versus **46.2 seconds** for C1 (**2.21× speedup**).
- **Parameter vs. Compute Tradeoff:** While C5 contains more total parameters (10.08M vs. 7.68M) due to the local encoder and decoder networks, it executes over twice as fast because

quadratic attention costs dominate linear projections.

3.2 Accuracy and Edit Distance Tradeoffs

While C5 incurs a 0.87pp drop in bit-level accuracy (63.00% vs. 63.87%), it achieves a **4.5% lower Levenshtein distance** (578.2 vs. 605.4).

This highlights a key vulnerability of subword tokenization: an erroneous bit within a subword unit alters the entire decoded word token, incurring large string edit penalties. BLT's local decoder predicts individual byte distributions directly, confining prediction errors to localized byte edits and avoiding error propagation across token boundaries.

Evaluation Dimension	C1 (Subword Baseline)	C5 (Token-Free BLT)	Relative Impact
Peak GPU Memory	5,319 MB	1,402 MB	**−73.6%**
Epoch Runtime	46.2 s	20.9 s	**−54.8%**
Bit-Level Accuracy	63.87%	63.00%	−0.87 pp
Avg. Levenshtein Distance	605.4	578.2	**−4.5% (Better)**
Active Parameters	7.68M	10.08M	+31.2%

Table 4: System-level trade-offs between subword tokenization and Byte Latent Transformer.

4 Conclusion

Through this from-scratch implementation and ablation study, I identified the following insights:

1. **RMSNorm** provides a drop-in replacement for LayerNorm that preserves convergence while removing mean computation.
2. **GQA** successfully reduces KV parameter overhead and memory with negligible accuracy degradation.
3. **BLT** provides an effective alternative to subword tokenizers for long byte sequences: patch compression reduces peak memory by **73.6%** and doubles training speed while improving character-level Levenshtein distance.

Reproducibility Links

- **Weights & Biases Public Report:** https://api.wandb.ai/links/dnebhrajani-v/xwjp7e0c
- **Hugging Face Checkpoints:** https://huggingface.co/dnebh/transformer-scratch

References

[1] A. Vaswani et al., "Attention is All You Need," in *NeurIPS*, 2017.

[2] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv:2104.09864*, 2021.

[3] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," *arXiv:2305.13245*, 2023.

[4] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *NeurIPS*, 2019.

[5] L. Yu et al., "Byte Latent Transformer: Patches Scale Better Than Tokens," *arXiv:2412.09871*, 2024.

[6] R. Sennrich et al., "Neural Machine Translation of Rare Words with Subword Units," in *ACL*, 2016.